

DOMAIN-DRIVEN DESIGN ANALYSIS

AI-Powered Multi-Tenant IVR Platform

*Complete Business Domain Model — Domains, Entities, Value Objects, Aggregates, Services, Business Rules
& Relationships*

Document Version: 1.0

Date: July 16, 2026

Status: Draft for Review — precedes database design

Prepared as part of a Graduation Project

Table of Contents

Table of Contents	2
Introduction	4
Step 1 — Business Domains	5
Step 2 — Entities by Domain.....	7
Identity & Access Management	7
Tenant Management.....	9
Organization Management	10
Telephony / SIP Gateway	11
IVR Design	12
IVR Execution (Runtime)	14
Media & Prompt Management	15
Queueing & Routing.....	15
Call Records & Recording	16
Reporting & Analytics.....	18
AI Assistant.....	19
Notification.....	20
Audit & Compliance	21
Deployment.....	21
Monitoring	22
Billing & Metering	24
Step 3 — Value Objects.....	25
Step 4 — Aggregates	26
Step 5 — Domain Services	32
Step 6 — Business Rules.....	34
Tenancy & Isolation.....	34
Organization	34
IVR Design & Versioning.....	34
Execution & Telephony	34
Recording & Compliance	35
AI Assistance.....	35
Access Control & Audit.....	35
Deployment.....	35

Billing (where enabled)	35
Step 7 — Relationships	36
Step 8 — Full Textual Domain Model.....	38
Step 9 — Review of the Domain Model.....	40
9.1 Missing Entities Identified During Review.....	40
9.2 Missing Services Identified During Review.....	40
9.3 Missing Business Rules Identified During Review	40
9.4 Architecture Improvements	40
9.5 Scalability Concerns.....	41
9.6 Maintainability Concerns	41
9.7 Future Expansion Opportunities	41

Introduction

This document performs a complete Domain-Driven Design (DDD) analysis of the AI-Powered Multi-Tenant IVR Platform, building on the prior Business Analysis and IEEE 830 SRS deliverables. It defines the platform's business domains, the entities and value objects within each, the aggregate boundaries and their invariants, the domain services that orchestrate behavior, the governing business rules, and the relationships that tie the model together. No SQL, ER diagrams, or source code are included — this is the conceptual domain model intended to precede and inform database design.

Step 1 — Business Domains

Sixteen business domains have been identified. Each represents a distinct area of responsibility with its own language, entities, and rules of change.

- 1. Identity & Access Management** — Authentication, authorization, roles, permissions, and session lifecycle for every human user of the platform, scoped to a tenant.
- 2. Tenant Management** — The lifecycle of a company (tenant) on the platform: registration, approval, plan/entitlement, suspension, and offboarding. The root isolation boundary of the entire system.
- 3. Organization Management** — A tenant's internal structure: departments, employees, and the SIP extensions assigned to them.
- 4. Telephony / SIP Gateway** — Physical/logical telephony connectivity: SIP trunks and DIDs (phone numbers) that connect the outside PSTN/SIP world to a tenant's platform configuration.
- 5. IVR Design** — Authoring concern: building, validating, and versioning IVR flows as a graph of nodes. Never touches a live call.
- 6. IVR Execution (Runtime)** — Real-time concern: interpreting a deployed Flow Version for a specific live call, tracking call state, and driving Asterisk call control.
- 7. Media & Prompt Management** — Voice prompt assets (uploaded or AI/TTS-generated) used by IVR nodes for playback to callers.
- 8. Queueing & Routing** — Holding callers awaiting an agent, applying routing strategies, and tracking agent availability and membership.
- 9. Call Records & Recording** — The historical, evidentiary record of calls: CDRs, recordings, voicemail, and consent capture.
- 10. Reporting & Analytics** — Aggregation and presentation of operational data derived from Call Records and Execution telemetry.
- 11. AI Assistant** — Orchestration of calls to an external LLM/TTS provider for flow generation, prompt generation, workflow improvement suggestions, and report summarization. Administrative only — never touches a live call.
- 12. Notification** — Delivery of alerts and messages to platform users (tenant admins, agents, super admins) across channels (email, in-app, SMS).
- 13. Audit & Compliance** — Tamper-evident tracking of configuration changes and consent, supporting regulatory and contractual compliance obligations.
- 14. Deployment** — The controlled process of making a published IVR Flow Version live on a DID, including deployment history, environment targeting, and rollback.
- 15. Monitoring** — Platform and tenant-level operational health: metrics, health checks, and alerting on threshold breaches (e.g., SLA breaches, elevated failure rate).

16. Billing & Metering — Usage measurement against a tenant's plan and invoice generation. Present to support the SaaS commercial model; may be deferred in an initial release without affecting the rest of the domain model.

Step 2 — Entities by Domain

Identity & Access Management

User

Purpose: Represents an authenticated human identity within a tenant (or, for Super Admin, the platform itself).

Responsibilities:

- Owns login credentials
- Carries role assignment(s)
- Is the actor referenced by audit records

Key Attributes:

- userId (identity)
- tenantId (null for Super Admin)
- email
- passwordHash
- status (Invited/Active/Locked/Deactivated)
- mfaEnabled
- lastLoginAt

Lifecycle: Invited → Active → (Locked ⇌ Active) → Deactivated. Deactivation is soft (retained for audit trail integrity).

Relationships:

- belongs to at most one Tenant
- holds one or more Role references
- has many Session instances
- optionally linked 1:1 to an Employee

Role

Purpose: Names a set of permissions that can be assigned to Users.

Responsibilities:

- Groups Permission value objects into a coherent capability set
- Distinguishes platform-defined roles from tenant-defined custom roles

Key Attributes:

- roleId
- tenantId (null for platform-defined roles)
- name
- permissions (list of Permission VOs)

- scope (Platform/Tenant/Department)

Lifecycle: Created (platform seed or Tenant Admin-defined) → Active → Retired (only if unassigned).

Relationships:

- assigned to many Users (many-to-many via assignment)
- scoped to one Tenant, or platform-wide if tenantId is null

Session

Purpose: Represents one authenticated login instance.

Responsibilities:

- Tracks session validity window
- Carries device/context metadata for security review

Key Attributes:

- sessionId
- userId
- issuedAt
- expiresAt
- ipAddress
- userAgent
- revoked (bool)

Lifecycle: Issued at login → active until expiry or explicit logout/revocation.

Relationships:

- belongs to exactly one User

ApiKey

Purpose: Represents a tenant-issued credential for external system integration (e.g., a tenant's CRM calling the platform's public API).

Responsibilities:

- Authenticates non-interactive API callers
- Scopes access to specific permissions/endpoints

Key Attributes:

- apiKeyId
- tenantId
- keyHash
- label
- scopes
- createdBy (User)
- status (Active/Revoked)

- lastUsedAt

Lifecycle: Generated → Active → Revoked (irreversible).

Relationships:

- belongs to one Tenant
- created by one User

Tenant Management

Tenant

Purpose: The isolation boundary of the platform; represents one customer company.

Responsibilities:

- Owns, directly or transitively, every tenant-scoped entity in the system
- Carries global configuration defaults (timezone, branding, default retention policy)

Key Attributes:

- tenantId (identity)
- companyName
- industryType
- status (Pending/Active/Suspended/Rejected/Offboarded)
- timezone
- brandingConfig
- defaultRetentionPolicy

Lifecycle: Pending → Active ⇌ Suspended → Offboarded/Rejected (terminal).

Relationships:

- has one Subscription
- root of every tenant-scoped aggregate (Department, Employee, IVRFlow, DID, etc.) by reference

Subscription

Purpose: Defines what a Tenant is entitled to use.

Responsibilities:

- Caps resource usage (extensions, queues, AI calls, storage)
- Drives feature-flag availability

Key Attributes:

- subscriptionId
- tenantId
- planName
- limits (map of resource → quota)
- billingCycle

- renewsAt

Lifecycle: Assigned at tenant activation → upgraded/downgraded → expired/cancelled.

Relationships:

- belongs to exactly one Tenant

Organization Management

Department

Purpose: A logical subdivision within a tenant's organization.

Responsibilities:

- Groups Employees and Call Queues
- May override tenant-level business hours

Key Attributes:

- departmentId
- tenantId
- name
- businessHoursOverride (optional)
- status (Active/Archived)

Lifecycle: Created → Active → Archived (blocked while it has active Employees/Queues).

Relationships:

- belongs to one Tenant
- has many Employees
- has many Call Queues

Employee

Purpose: A person who handles calls on behalf of the tenant.

Responsibilities:

- Represents agent availability state
- Carries skill tags used for routing

Key Attributes:

- employeeId
- tenantId
- departmentId(s)
- name
- status (Available/Busy/Offline/Deactivated)
- skills (list of Skill VOs)
- linkedUserId (optional)

Lifecycle: Onboarded → Active (Available⇌Busy⇌Offline) → Offboarded (deactivates associated extension and queue memberships).

Relationships:

- belongs to one or more Departments
- has zero or one active SIP Extension
- member of zero or more Call Queues (via QueueMembership)
- optionally linked 1:1 to a User

SIP Extension

Purpose: The telephony endpoint an Employee's device/softphone registers to.

Responsibilities:

- Holds SIP registration credentials
- Represents the routable destination for Agent Transfer

Key Attributes:

- extensionId
- tenantId
- extensionNumber
- sipCredentials (VO)
- registrationStatus
- status (Active/Deactivated)

Lifecycle: Provisioned (unassigned) → Assigned to Employee → Active → Deactivated (on employee offboarding or explicit revocation).

Relationships:

- belongs to one Tenant
- assigned to at most one Employee at a time

Telephony / SIP Gateway

SIP Trunk

Purpose: The connection between Asterisk and the outside telephony network (PSTN/SIP provider).

Responsibilities:

- Carries inbound/outbound signaling and media capacity
- Enforces concurrency limits

Key Attributes:

- trunkId
- tenantId (null if platform-shared)
- provider

- credentials (VO)
- maxConcurrentCalls
- status (Active/Disabled)

Lifecycle: Configured → Active → Disabled.

Relationships:

- may be shared platform-wide or dedicated to one Tenant
- owns many DIDs

DID (Phone Number)

Purpose: The public phone number a caller dials to reach a tenant.

Responsibilities:

- Resolves an inbound call to a Tenant and its currently deployed Flow Version

Key Attributes:

- didId
- tenantId
- number (PhoneNumber VO)
- trunkId
- status (Active/Released)

Lifecycle: Purchased/porting → Assigned to Tenant → Mapped to a deployed Flow Version → Released.

Relationships:

- belongs to one Tenant
- belongs to one SIP Trunk
- mapped to at most one active Flow Version at a time (via Deployment)

IVR Design

IVRFlow

Purpose: The named, long-lived design-time container a Tenant Admin edits; represents intent independent of any single deployment.

Responsibilities:

- Groups all historical Flow Versions under one identity
- Tracks the current draft pointer

Key Attributes:

- flowId
- tenantId
- name
- description

- currentDraftVersionId (optional)
- status (Active/Retired)

Lifecycle: Created → iterated via drafts → (has 0..N published versions) → Retired.

Relationships:

- belongs to one Tenant
- has many Flow Versions

FlowVersion

Purpose: An immutable, numbered snapshot of a flow's node graph; the unit of validation and deployment.

Responsibilities:

- Holds the complete node graph for that snapshot
- Is the only artifact the Execution engine is permitted to read

Key Attributes:

- flowVersionId
- flowId
- versionNumber (VO)
- status (Draft/Validated/Published/Archived)
- createdBy
- createdAt
- validationResult

Lifecycle: Draft → Validated → Published (immutable from here) → superseded → Archived.

Relationships:

- belongs to one IVRFlow
- composed of many Nodes and Connections
- may be the deployed target of one or more DIDs (via Deployment)

Node

Purpose: A single configurable step in the workflow graph.

Responsibilities:

- Encapsulates one behavior type (Greeting, Menu, Queue, API Request, etc.) and its type-specific configuration

Key Attributes:

- nodeId
- flowVersionId
- nodeType (discriminator)
- configuration (NodeConfiguration VO, shape varies by type)
- canvasPosition

Lifecycle: Added to a draft → configured → connected → immutable once the owning Flow Version is published.

Relationships:

- belongs to one Flow Version
- source/target of many Connections

Connection

Purpose: A directed edge between two nodes, optionally conditioned on an outcome.

Responsibilities:

- Determines execution order and branching logic

Key Attributes:

- connectionId
- flowVersionId
- sourceNodeId
- targetNodeId
- conditionLabel (e.g., DTMF digit, 'success'/'failure')

Lifecycle: Created during design → immutable once the owning Flow Version is published.

Relationships:

- belongs to one Flow Version
- references exactly two Nodes (source and target) within the same Flow Version

IVR Execution (Runtime)

CallSession

Purpose: Represents one live or completed phone call end-to-end, and the current position of flow execution.

Responsibilities:

- Tracks the caller's journey through the deployed Flow Version
- Coordinates with Asterisk for real-time call control

Key Attributes:

- callSessionId
- tenantId
- didId
- flowVersionId
- callerNumber (PhoneNumber VO)
- currentNodeId
- startedAt

- endedAt
- disposition (CallDisposition VO)

Lifecycle: Ringing → InProgress (executing nodes) → ConnectedToAgent/Queue/Voicemail → Ended. On Ended, a CDR is generated.

Relationships:

- belongs to one Tenant
- executes against one Flow Version
- produces exactly one CDR
- may produce a Recording and/or a Voicemail
- emits many Analytics Events

Media & Prompt Management

VoicePrompt

Purpose: An audio asset played to callers.

Responsibilities:

- Stores or references playable audio for Greeting/Playback nodes

Key Attributes:

- promptId
- tenantId
- name
- locale
- source (Uploaded/AIGenerated)
- audioFileRef
- durationSeconds
- status (Active/Archived)

Lifecycle: Created (upload or AI generation) → used in flows → Archived (never hard-deleted while referenced by any Flow Version, published or not).

Relationships:

- belongs to one Tenant
- referenced by zero or more Nodes

Queueing & Routing

CallQueue

Purpose: Holds callers awaiting the next available agent.

Responsibilities:

- Applies a routing strategy
- Tracks wait time against a configured SLA
- Defines overflow behavior

Key Attributes:

- queueId
- tenantId
- departmentId (optional)
- name
- strategy (RoundRobin/LongestIdle/SkillsBased)
- maxWaitSeconds
- overflowAction
- status (Active/Paused/Archived)

Lifecycle: Created → Active ⇌ Paused → Archived (blocked while it has active memberships referenced by a published flow).

Relationships:

- belongs to one Tenant/Department
- has many Employees via Queue Membership
- referenced by zero or more Queue Nodes

QueueMembership

Purpose: Represents an Employee's participation in a Queue, with routing-relevant attributes.

Responsibilities:

- Carries priority/skill weighting specific to that queue

Key Attributes:

- queueMembershipId
- queueId
- employeeId
- priority
- addedAt

Lifecycle: Added → Active → Removed.

Relationships:

- links exactly one Employee to exactly one Call Queue

Call Records & Recording

CallDetailRecord (CDR)

Purpose: The permanent, immutable historical record of a completed call.

Responsibilities:

- Provides the ground truth for Reporting

Key Attributes:

- cdrId
- tenantId
- callSessionId
- startedAt
- endedAt
- durationSeconds
- disposition
- pathSummary (nodes visited)
- queueId (optional)
- employeeId (optional)

Lifecycle: Created once, at call end. Immutable thereafter. Archived per retention policy (not deleted, moved to cold storage where applicable).

Relationships:

- derived from exactly one Call Session
- belongs to one Tenant

Recording

Purpose: The audio artifact of a recorded call.

Responsibilities:

- Stores/retrieves call audio under access control
- Enforces retention expiry

Key Attributes:

- recordingId
- tenantId
- callSessionId
- audioFileRef
- durationSeconds
- consentRecordId (optional)
- retentionExpiresAt

Lifecycle: Recorded → Stored → (Accessed, logged) → Purged at retention expiry.

Relationships:

- belongs to one Call Session
- belongs to one Tenant

- optionally linked to one Consent Record

Voicemail

Purpose: A message left by a caller when no agent was available.

Responsibilities:

- Stores audio and read/unread state
- Notifies the assigned recipient

Key Attributes:

- voicemailId
- tenantId
- callSessionId
- audioFileRef
- durationSeconds
- recipientEmployeeId/DepartmentId
- status (New/Listened/Archived)

Lifecycle: Recorded → New → Listened → Archived/Deleted.

Relationships:

- belongs to one Call Session
- assigned to one Employee or Department

ConsentRecord

Purpose: Tracks whether/when a caller consented to being recorded.

Responsibilities:

- Supports compliance reporting for regulated tenants (banks, hospitals)

Key Attributes:

- consentRecordId
- callSessionId
- consentGiven (bool)
- method (e.g., announcement played)
- capturedAt

Lifecycle: Created at call start → immutable.

Relationships:

- belongs to one Call Session

Reporting & Analytics

Report

Purpose: A generated or on-demand analytical view.

Responsibilities:

- Aggregates CDRs and Analytics Events into a consumable output

Key Attributes:

- reportId
- tenantId
- type
- dateRange (VO)
- generatedAt
- format (Dashboard/CSV/PDF)
- requestedBy

Lifecycle: Requested/Scheduled → Generated → Viewed/Exported → (retained/deleted per policy).

Relationships:

- belongs to one Tenant
- built from many CDRs and Analytics Events

AnalyticsEvent

Purpose: Fine-grained telemetry captured during flow execution.

Responsibilities:

- Feeds computation of flow drop-off, path popularity, and completion time

Key Attributes:

- eventId
- callSessionId
- nodeId
- eventType
- timestamp

Lifecycle: Emitted during execution → aggregated → optionally purged after aggregation per retention policy.

Relationships:

- belongs to one Call Session
- references one Node

AI Assistant

AIRequestJob

Purpose: Represents one interaction with the external LLM/TTS provider.

Responsibilities:

- Stores the prompt/input, the response/output, and the resulting draft artifact
- Tracks cost/usage for metering

Key Attributes:

- jobId
- tenantId
- requestedBy
- requestType (FlowGeneration/PromptGeneration/ImprovementSuggestion/ReportSummary)
- inputPayload
- outputPayload
- status (Pending/Completed/Failed)
- tokensOrCostUsed

Lifecycle: Requested → Processing → Completed/Failed → (its output accepted or discarded by a human).

Relationships:

- belongs to one Tenant and one requesting User
- may produce a draft Flow Version and/or a Voice Prompt

Notification

NotificationTemplate

Purpose: A reusable, parameterized message definition (e.g., 'tenant suspended', 'new voicemail').

Responsibilities:

- Standardizes message content and channel per event type

Key Attributes:

- templateId
- eventType
- channel (Email/InApp/SMS)
- subjectTemplate
- bodyTemplate

Lifecycle: Defined (platform-seeded, rarely tenant-customized) → Active → Retired.

Relationships:

- referenced by many Notifications

Notification

Purpose: A single message instance delivered (or pending delivery) to a recipient.

Responsibilities:

- Tracks delivery state for one recipient/event occurrence

Key Attributes:

- notificationId
- tenantId
- recipientUserId
- templateId
- renderedContent
- status (Pending/Sent/Failed/Read)
- createdAt

Lifecycle: Created → Sent → Delivered/Failed → (Read, for in-app).

Relationships:

- belongs to one recipient User
- instantiated from one Notification Template

Audit & Compliance

AuditLogEntry

Purpose: Records who changed what, when, across the platform.

Responsibilities:

- Provides traceability for configuration changes for compliance and troubleshooting

Key Attributes:

- auditLogId
- tenantId (null for platform-level actions)
- actorUserId
- action
- targetEntityType
- targetEntityId
- beforeSnapshot (optional)
- afterSnapshot (optional)
- timestamp

Lifecycle: Append-only; retained per compliance policy; never updated or deleted before retention expiry.

Relationships:

- references one actor User and one target entity (polymorphic reference)

Deployment

Deployment

Purpose: Records the act of making a published Flow Version live on a specific DID.

Responsibilities:

- Establishes the DID→FlowVersion mapping used by call routing
- Preserves deployment history for rollback and audit

Key Attributes:

- deploymentId
- tenantId
- didId
- flowVersionId
- environment (Production/Sandbox)
- deployedBy
- deployedAt
- status (Active/Superseded/RolledBack)

Lifecycle: Created (a deploy action) → Active → Superseded by a newer Deployment or RolledBack to a prior one.

Relationships:

- belongs to one Tenant
- references one DID and one Flow Version

DeploymentEnvironment

Purpose: Represents a target context for deployment (e.g., Production vs. Sandbox/staging), enabling safe pre-production testing.

Responsibilities:

- Scopes which DIDs/test numbers a deployment affects

Key Attributes:

- environmentId
- tenantId
- name
- type (Production/Sandbox)

Lifecycle: Created (Production exists by default per tenant; Sandbox optional) → Active.

Relationships:

- belongs to one Tenant
- target of many Deployments

Monitoring**HealthCheck**

Purpose: Represents the operational status of a platform component at a point in time.

Responsibilities:

- Feeds the platform status view used by Super Admin and internal alerting

Key Attributes:

- healthCheckId
- component (WebApp/TelephonyLayer/AllIntegration/Database)
- status (Up/Degraded/Down)
- checkedAt
- details

Lifecycle: Recorded periodically (polling) → superseded by the next check.

Relationships:

- not tenant-scoped; platform-level

SystemMetric

Purpose: A time-series operational measurement (e.g., concurrent calls, queue wait time, API latency).

Responsibilities:

- Supports dashboards and alert-rule evaluation

Key Attributes:

- metricId
- tenantId (optional, null for platform-wide metrics)
- metricName
- value
- timestamp

Lifecycle: Emitted continuously → aggregated/rolled up → purged per retention policy.

Relationships:

- optionally scoped to one Tenant

AlertRule

Purpose: Defines a threshold condition that, when breached, should notify an operator.

Responsibilities:

- Evaluates SystemMetric streams against a configured threshold

Key Attributes:

- alertRuleId
- tenantId (optional)
- metricName
- condition (e.g., > threshold for N minutes)
- severity
- notifyRoles

Lifecycle: Defined → Active ⇌ Muted → Retired.

Relationships:

- evaluates one or more System Metrics
- triggers Notifications when breached

Billing & Metering**UsageRecord**

Purpose: Meters consumption against the tenant's plan.

Responsibilities:

- Feeds invoicing and enforces plan limits

Key Attributes:

- usageRecordId
- tenantId
- metricType (Minutes/AlCalls/Storage/Extensions)
- quantity
- period

Lifecycle: Accumulated continuously → reset per billing cycle.

Relationships:

- belongs to one Tenant

Invoice

Purpose: The billing document for a tenant's usage/subscription in a period.

Responsibilities:

- Summarizes charges for a billing period

Key Attributes:

- invoiceId
- tenantId
- period
- lineItems
- total (Money VO)
- paymentStatus

Lifecycle: Generated → Sent → Paid/Overdue.

Relationships:

- belongs to one Tenant

Step 3 — Value Objects

Value Objects are defined entirely by their attributes, are immutable once created, carry no independent identity, and are freely interchangeable when their values are equal. The following have been identified across the domain model:

PhoneNumber — Defined entirely by its value (E.164 string) with validation rules; two PhoneNumbers with the same digits are interchangeable and have no independent identity or lifecycle.

EmailAddress — A validated string with equality by value; carries no identity of its own beyond the User it's attached to.

SipCredentials — An immutable (username, secret, realm) tuple generated once at provisioning; replacing credentials means issuing a new value, not mutating the old one.

VersionNumber — A simple, comparable, immutable value (e.g., integer or semantic triple) with no lifecycle of its own — it only ever describes a Flow Version.

BusinessHoursWindow — A set of day/time ranges plus a timezone; two identical windows are indistinguishable and interchangeable, with no persistent identity.

DateRange — A (start, end) pair used to scope reports; entirely defined by its two bounds, immutable once constructed.

Money — An (amount, currency) pair with standard value-object equality and arithmetic; never mutated in place.

Permission — A named capability string (e.g., 'ivr.flow.publish'); identical permissions are always interchangeable regardless of which Role holds them.

Skill — A label/tag (e.g., 'Arabic', 'Billing') attached to Employees and used in Queue routing; equality is by name, with no independent lifecycle.

RetentionPolicy — An immutable (durationDays, action) rule attached to Tenant/Recording configuration; changing it means replacing the value, not editing history.

CallDisposition — An enumerated outcome (Answered/Abandoned/Voicemail/Failed) describing how a call ended; a closed, immutable set of values with no identity.

NodeConfiguration — The type-specific configuration payload of a Node (e.g., a Menu's digit map, an API Request's endpoint and timeout); it is wholly replaced on edit rather than field-mutated, and is meaningless outside the Node that holds it.

GeoTimezone — An IANA timezone identifier used for schedule evaluation; a pure value with global, shared meaning.

ConsentStatus — A small enumerated value (Given/Declined/NotRequired) describing the outcome of a consent capture, with no lifecycle beyond the ConsentRecord that holds it.

Step 4 — Aggregates

Each aggregate defines a transactional consistency boundary. External references to entities inside an aggregate are made only through the aggregate root's identity; nothing outside the boundary is permitted to hold a direct reference to a child entity.

Tenant Aggregate

Aggregate Root: Tenant

Child Entities / Contained Elements: Subscription

Business Rules:

- A Tenant must have exactly one active Subscription at a time.
- A Tenant cannot be Active without having passed through Pending approval.

Consistency Rules:

- Changes to plan limits and tenant status must be applied atomically so a suspended tenant can never simultaneously show an active Subscription entitlement.

User Aggregate

Aggregate Root: User

Child Entities / Contained Elements: Session (references only; sessions are managed transactionally on their own but conceptually owned by the User)

Business Rules:

- A User belongs to at most one Tenant.
- A locked User cannot create a new Session.

Consistency Rules:

- Role assignment changes must not leave a User with zero roles inside an active tenant if at least one role is required for basic access.

Role Aggregate

Aggregate Root: Role

Child Entities / Contained Elements: Permission (value objects, held by value inside the Role)

Business Rules:

- A tenant-scoped custom Role may not grant a Permission outside the tenant's plan entitlements.

Consistency Rules:

- The Permission set of a Role is modified as a whole (replace-the-set), not by ad hoc partial mutation, to avoid inconsistent intermediate states visible to concurrent authorization checks.

Department Aggregate

Aggregate Root: Department

Child Entities / Contained Elements: None (single-entity aggregate)

Business Rules:

- A Department cannot be archived while it has Active Employees or Call Queues.

Consistency Rules:

- Business-hours override, if present, must always resolve to a complete, valid schedule (no partial override states).

Employee Aggregate

Aggregate Root: Employee

Child Entities / Contained Elements: QueueMembership (references; memberships are tracked by the Queue Aggregate but visible from Employee)

Business Rules:

- Deactivating an Employee cascades to deactivate their SIP Extension and remove all Queue Memberships.

Consistency Rules:

- Employee status and extension registration status must never contradict each other for longer than the propagation of a single transaction.

SIPExtension Aggregate

Aggregate Root: SIPExtension

Child Entities / Contained Elements: None (single-entity aggregate)

Business Rules:

- An extension may be assigned to at most one Employee at a time.
- Extension numbers are unique within a Tenant.

Consistency Rules:

- Reassignment of an extension to a new Employee must atomically release it from the previous Employee.

SIPTrunk Aggregate

Aggregate Root: SIPTrunk

Child Entities / Contained Elements: None (single-entity aggregate)

Business Rules:

- A disabled trunk must not accept new calls but may complete calls already in progress.

Consistency Rules:

- maxConcurrentCalls must never be exceeded; enforcement happens synchronously at call admission time.

DID Aggregate

Aggregate Root: DID

Child Entities / Contained Elements: None (single-entity aggregate)

Business Rules:

- A DID belongs to exactly one Tenant.
- A DID may be actively mapped to at most one Flow Version at a time (enforced via the Deployment Aggregate).

Consistency Rules:

- Releasing a DID must first remove any active Deployment referencing it.

IVRFlow Aggregate

Aggregate Root: IVRFlow

Child Entities / Contained Elements: None (single-entity aggregate)

Business Rules:

- An IVRFlow belongs to exactly one Tenant.
- Only one Flow Version of a given Flow may be in Draft status at a time.

Consistency Rules:

- The currentDraftVersionId must always reference a Flow Version that actually belongs to this Flow.

FlowVersion Aggregate

Aggregate Root: FlowVersion

Child Entities / Contained Elements: Node; Connection

Business Rules:

- A published Flow Version and its Nodes/Connections are immutable.
- Every Node except terminal types must have at least one outgoing Connection.
- Every Flow Version must have exactly one entry Node.

Consistency Rules:

- Nodes and Connections are only ever persisted together with their owning Flow Version's validation pass; a Flow Version cannot be marked Published while containing a dangling Connection referencing a non-existent Node.

CallQueue Aggregate

Aggregate Root: CallQueue

Child Entities / Contained Elements: QueueMembership

Business Rules:

- A Queue used by any published Flow Version must retain at least one active member Employee, or its overflow action must be reachable without a member.
- Queue strategy determines the tie-break rule among equally eligible agents.

Consistency Rules:

- Adding/removing a QueueMembership must be atomic with respect to concurrent routing decisions — an Employee mid-removal must not receive a new call.

CallSession Aggregate

Aggregate Root: CallSession

Child Entities / Contained Elements: AnalyticsEvent (append-only stream, conceptually owned by the session during its lifetime)

Business Rules:

- A CallSession executes against exactly one immutable Flow Version for its entire lifetime, even if a newer version is published mid-call.
- A CallSession must transition to Ended exactly once.

Consistency Rules:

- currentNodeId must always reference a Node that belongs to the session's bound flowVersionId.
- Ending a session and generating its CDR must be treated as a single logical transaction boundary.

Recording Aggregate

Aggregate Root: Recording

Child Entities / Contained Elements: None (single-entity aggregate)

Business Rules:

- A Recording's retentionExpiresAt must never exceed the Tenant's configured maximum retention.

Consistency Rules:

- Purge jobs must remove the audio file and the metadata row together; a Recording row must never outlive its file, nor vice versa.

Voicemail Aggregate

Aggregate Root: Voicemail

Child Entities / Contained Elements: None (single-entity aggregate)

Business Rules:

- A Voicemail must have exactly one recipient (Employee or Department).

Consistency Rules:

- Status transition to Listened may only occur after successful playback confirmation from the client.

CDR Aggregate

Aggregate Root: CallDetailRecord

Child Entities / Contained Elements: None (single-entity aggregate)

Business Rules:

- A CDR is immutable once created; corrections require a compensating record.

Consistency Rules:

- A CDR is created exactly once per CallSession, enforced by a uniqueness constraint on callSessionId.

Report Aggregate

Aggregate Root: Report

Child Entities / Contained Elements: None (single-entity aggregate)

Business Rules:

- A generated Report's data must never span more than one Tenant.

Consistency Rules:

- A Report's snapshot data, once generated, does not change even if underlying CDRs are later archived — it reflects the state at generation time.

AIRequestJob Aggregate

Aggregate Root: AIRequestJob

Child Entities / Contained Elements: None (single-entity aggregate)

Business Rules:

- An AIRequestJob's output may only be materialized into a draft artifact (Flow Version draft or Voice Prompt); it can never directly modify a published artifact.

Consistency Rules:

- Job status must move monotonically Pending → Processing → (Completed | Failed); no backward transitions.

AuditLogEntry Aggregate

Aggregate Root: AuditLogEntry

Child Entities / Contained Elements: None (single-entity aggregate)

Business Rules:

- Every mutating operation on a tenant-scoped aggregate must produce at least one corresponding AuditLogEntry.

Consistency Rules:

- Audit entries are append-only; no update or delete operation is permitted before retention expiry.

Deployment Aggregate

Aggregate Root: Deployment

Child Entities / Contained Elements: None (single-entity aggregate)

Business Rules:

- Creating a new Active Deployment for a DID must supersede the previously Active Deployment for that same DID.

Consistency Rules:

- Exactly one Deployment per DID may be in Active status at any time — enforced as an invariant of the aggregate boundary, not left to application-level discipline alone.

Invoice Aggregate

Aggregate Root: Invoice

Child Entities / Contained Elements: None (single-entity aggregate)

Business Rules:

- An Invoice's line items must reconcile against the UsageRecords for its stated period.

Consistency Rules:

- Once marked Sent, an Invoice's line items become immutable; corrections require a credit note (future enhancement), not in-place edits.

Step 5 — Domain Services

Domain services encapsulate behavior that does not naturally belong to a single entity or aggregate — typically because it coordinates across multiple aggregates or an external system.

Service	Responsibility
AuthenticationService	Verifies credentials, issues and revokes Sessions, enforces password/MFA policy and account lockout.
AuthorizationService	Evaluates whether an authenticated User's Role(s) permit a requested action, and enforces tenant scoping on every check.
TenantProvisioningService	Orchestrates tenant registration, approval, suspension, and offboarding, including cascading effects (e.g., disabling DIDs on suspension).
OrganizationManagementService	Coordinates Department and Employee lifecycle operations, including cascading deactivation rules.
ExtensionProvisioningService	Generates SIP credentials, provisions extensions, and keeps Asterisk's PJSIP endpoint configuration in sync with platform state.
SipTrunkManagementService	Manages trunk configuration and capacity, and enforces concurrent-call admission control.
IVRDesignService	Backs the visual builder: creating/editing drafts, managing Node/Connection CRUD within a draft Flow Version.
FlowValidationService	Validates a draft Flow Version's graph structure (entry node, reachability, required fallback paths) before it can be published.
FlowVersioningService	Publishes a validated draft into an immutable Flow Version, and manages rollback to a prior version.
DeploymentService	Maps a published Flow Version to a DID within an environment, superseding any prior active Deployment for that DID.
CallRoutingService	Resolves an inbound call's DID to its Tenant and active Deployment/Flow Version, and creates the CallSession.
ExecutionEngineService	Interprets the Flow Version node-by-node for a live CallSession, issuing Asterisk call-control commands and following fallback paths on node failure.
QueueRoutingService	Selects the next Employee for a waiting caller per the Queue's strategy, and triggers overflow action on SLA breach.
RecordingService	Starts/stops call recording, stores audio, and enforces retention/consent rules.
VoicemailService	Captures voicemail audio, persists the Voicemail entity, and triggers recipient notification.
ConsentService	Determines whether consent capture is required for a given call and records the outcome.
ReportingService	Aggregates CDRs and Analytics Events into requested Report views, enforcing tenant/department scoping on access.

Service	Responsibility
AnalyticsAggregationService	Rolls up raw Analytics Events into flow-level metrics (drop-off rate, path popularity, completion time).
AIOrchestrationService	Sends scoped, sanitized requests to the external LLM/TTS provider, tracks AIRequestJob state, and materializes results as drafts only.
NotificationService	Renders NotificationTemplates and delivers Notifications across configured channels, tracking delivery status.
AuditService	Captures AuditLogEntry records for every auditable mutation across all domains.
RetentionPolicyService	Applies configured RetentionPolicy values to purge Recordings, Voicemail, and Analytics Events on schedule.
MonitoringService	Collects HealthChecks and SystemMetrics, evaluates AlertRules, and triggers Notifications on breach.
BillingService	Aggregates UsageRecords per billing cycle and generates Invoices.

Step 6 — Business Rules

Business rules are grouped by concern area. Together with the aggregate consistency rules in Step 4, these form the complete set of invariants the application and, later, the database must enforce.

Tenancy & Isolation

- Every tenant-scoped entity belongs to exactly one Tenant; cross-tenant references are prohibited.
- A User can authenticate into only the Tenant(s) they belong to; Super Admin accounts are not tenant-scoped.
- A suspended Tenant's DIDs must reject new inbound calls, but in-progress CallSessions are allowed to complete.
- Data access at the persistence layer must always filter by tenantId; there is no code path that queries across tenants for business data.

Organization

- An Employee belongs to one Tenant and may belong to multiple Departments.
- A Department cannot be archived while it has active Employees or active Call Queues.
- An Extension may belong to at most one Employee at a time, and extension numbers are unique within a Tenant.
- Deactivating an Employee cascades to deactivate their Extension and remove all Queue Memberships.

IVR Design & Versioning

- An IVRFlow belongs to exactly one Tenant.
- Only one Draft Flow Version may exist per Flow at a time.
- A published Flow Version is immutable; further changes require a new draft and a new published version.
- Every Flow Version must have exactly one entry Node and every non-terminal Node must have at least one outgoing Connection.
- A Flow Version with unresolved validation errors cannot be published or deployed.
- A DID may be actively mapped to at most one Flow Version at a time.

Execution & Telephony

- A CallSession is bound to exactly one Flow Version for its entire duration, even if a newer version is published mid-call.
- A failed node (e.g., API Request timeout) must follow its configured fallback path rather than terminating the call.
- A Queue used by a published flow must always have a reachable outcome, whether via an available Employee or its overflow action.
- An Agent Transfer target that is unreachable must follow the flow's configured fallback path.

- Every completed call produces exactly one immutable CDR.

Recording & Compliance

- Recording requires either tenant-wide policy enablement or an explicit Recording node in the flow.
- Where a Tenant's configuration requires it, a consent announcement must be played and a ConsentRecord captured before recording begins.
- Recordings and Voicemail must be purged automatically at the expiry of their RetentionPolicy.
- Access to Recordings and Voicemail is restricted to authorized roles and is itself audit-logged.

AI Assistance

- AI-generated or AI-suggested artifacts (flows, prompts, recommendations) are always created as drafts and require explicit human review and approval before taking effect in a published state.
- Only aggregated, tenant-scoped data may be sent to the external AI provider — never raw recordings or personally identifying caller data.
- Every AI interaction is logged as an AIRequestJob for traceability and usage metering.

Access Control & Audit

- Every protected action is checked against the acting User's Role(s) and Tenant scope before execution, server-side.
- A tenant-scoped custom Role may not grant a Permission outside the Tenant's plan entitlements.
- Every mutating operation on a tenant-scoped aggregate produces a corresponding AuditLogEntry.
- Audit log entries are append-only and are never modified or deleted before their retention period expires.

Deployment

- Creating a new Active Deployment for a DID supersedes the previous Active Deployment for that DID.
- A Sandbox environment deployment must never affect a Production DID mapping.
- Rollback restores a prior Flow Version as the active Deployment without deleting the newer version's history.

Billing (where enabled)

- UsageRecords accumulate continuously and reset at each billing cycle boundary.
- An Invoice's line items must reconcile against the UsageRecords for its stated period and become immutable once sent.

Step 7 — Relationships

The table below classifies every significant relationship in the domain model by cardinality and structural type (association, aggregation, composition, or — in one deliberate case — polymorphism in place of inheritance).

Relationship	Cardinality	Type & Notes
Tenant — Subscription	One-to-One	Composition: a Subscription has no meaning or lifecycle outside its owning Tenant.
Tenant — Department	One-to-Many	Aggregation: Departments are tenant-scoped but are independent aggregates with their own identity and lifecycle.
Tenant — User	One-to-Many	Aggregation.
Tenant — DID	One-to-Many	Aggregation.
Tenant — IVRFlow	One-to-Many	Aggregation.
Department — Employee	One-to-Many (Employee may belong to multiple Departments → effectively Many-to-Many)	Aggregation; modeled as many-to-many if multi-department assignment is enabled, one-to-many if simplified.
Employee — SIP Extension	One-to-One (at a given time)	Composition-like: an Extension's assignment is tightly bound to one Employee, though the Extension entity itself outlives any single assignment.
Employee — CallQueue (via QueueMembership)	Many-to-Many	Association entity (QueueMembership) carries per-pair attributes (priority).
IVRFlow — FlowVersion	One-to-Many	Composition: a Flow Version has no meaning outside its owning Flow.
FlowVersion — Node	One-to-Many	Composition: Nodes cannot exist outside a Flow Version and are deleted/archived with it.
FlowVersion — Connection	One-to-Many	Composition.
Node — Connection	One-to-Many (as source), One-to-Many (as target)	Aggregation within the Flow Version boundary; a Connection references exactly two Nodes.
DID — Deployment	One-to-Many (historical), effectively One-to-One (active)	Aggregation: history is retained, but only one Deployment per DID is Active at a time.
FlowVersion — Deployment	One-to-Many	Aggregation: a single Flow Version may be deployed to multiple DIDs simultaneously.
CallSession — FlowVersion	Many-to-One	Association: many calls execute against the same immutable Flow Version.

Relationship	Cardinality	Type & Notes
CallSession — CDR	One-to-One	Composition-like derivation: exactly one CDR is generated per session, immutable and permanent.
CallSession — Recording	One-to-Zero-or-One	Aggregation; recording is conditional per business rules.
CallSession — Voicemail	One-to-Zero-or-One	Aggregation; conditional.
CallSession — AnalyticsEvent	One-to-Many	Composition: events have no meaning outside the session that produced them.
CallQueue — Employee (via QueueMembership)	Many-to-Many	See Employee—CallQueue above.
User — Role	Many-to-Many	Association: a User may hold multiple Roles; a Role may be assigned to many Users.
Role — Permission	One-to-Many, held by value	Composition: Permissions are Value Objects contained within the Role aggregate, not independently referenced entities.
AIRequestJob — FlowVersion (draft output)	Zero-or-One-to-One	Association: an AI job may produce at most one draft Flow Version.
AuditLogEntry — (any entity)	Many-to-One, polymorphic	Association via (entityType, entityId) reference; this is the one place a lightweight form of polymorphic association is used rather than entity inheritance.
Node type hierarchy (Greeting, Menu, Queue, API Request, ...)	Inheritance / polymorphism	Modeled as a single Node entity with a nodeType discriminator and a polymorphic NodeConfiguration value object, rather than a class-per-type table hierarchy — this keeps the graph engine uniform while still allowing type-specific behavior at the service layer (strategy pattern over the discriminator).

Step 8 — Full Textual Domain Model

The domain model is anchored by the Tenant Aggregate, which is the isolation boundary for the entire platform. Every other aggregate either belongs directly to a Tenant (via a `tenantId` reference) or, in the case of platform-level aggregates like `AuditLogEntry` (for Super Admin actions) and shared SIP Trunks, is explicitly marked as platform-scoped rather than tenant-scoped.

Within a Tenant, the Identity & Access domain (User, Role, Session, ApiKey) governs who may act and what they may do; every other domain's services consult `AuthorizationService` before permitting a mutation. The Organization Management domain (Department, Employee, SIP Extension) models the tenant's internal structure and is the bridge between the business organization and the Telephony domain: an Employee's SIP Extension is the routable endpoint that the IVR Execution engine transfers calls to.

The IVR Design domain (IVRFlow, FlowVersion, Node, Connection) is a purely authoring-time concern. A Tenant Admin — assisted optionally by the AI Assistant domain's `AIRequestJob`, which can propose a draft Flow Version from a natural-language description — iterates on a Draft Flow Version until `FlowValidationService` confirms it is structurally sound (single entry node, no orphan nodes, every path terminating correctly). `FlowVersioningService` then publishes it, producing an immutable Flow Version that can never again be edited in place.

The Deployment domain is the deliberate seam between design-time and run-time: a Deployment record binds one immutable Flow Version to one DID within an Environment (Production or Sandbox), and `DeploymentService` guarantees that only one Deployment is Active per DID at any moment. This seam exists specifically so that the IVR Execution domain never has to reason about drafts, validation state, or edit history — it reads only immutable, already-validated Flow Versions.

When a call arrives, the Telephony / SIP Gateway domain's DID resolves (via the active Deployment) to a Tenant and Flow Version, and `CallRoutingService` creates a `CallSession` — the aggregate root of the IVR Execution domain. `ExecutionEngineService` walks the Flow Version's Node graph, issuing Asterisk call-control commands for each node type (using the Media & Prompt Management domain's `VoicePrompt` assets for Greeting/Playback nodes, and the Queueing & Routing domain's `CallQueue/QueueMembership` for Queue nodes). Every node visited emits an `AnalyticsEvent`, and the `CallSession` is bound to its Flow Version for its entire lifetime — even if a newer version is published while the call is still in progress, ensuring a caller never experiences a mid-call change in logic.

When execution reaches a terminal outcome, the Call Records & Recording domain takes over: an immutable CDR aggregate is created exactly once per `CallSession`, and — depending on tenant policy and the flow's own Recording nodes — a Recording or Voicemail aggregate is produced, with a `ConsentRecord` captured first wherever the tenant's compliance configuration requires it.

The Reporting & Analytics domain consumes CDRs and `AnalyticsEvents` (never raw call audio) to build Report aggregates and computed flow-level metrics, which in turn feed back into the AI Assistant domain's improvement-suggestion capability — closing the loop from 'how did the flow actually perform' back to 'here's a proposed draft to improve it', always landing as a new Draft Flow Version subject to the same human-approval gate as any other AI output.

Cutting across every domain: the Audit & Compliance domain's `AuditLogEntry` aggregate records every mutating action platform-wide; the Notification domain delivers alerts (e.g., a new voicemail, a suspended

tenant, an SLA breach); and the Monitoring domain's HealthCheck/SystemMetric/AlertRule aggregates observe the platform's own operational health, independent of any single tenant's business data. The Billing & Metering domain, where enabled, observes UsageRecords accumulating from Telephony, AI, and Storage activity across the other domains and periodically produces Invoice aggregates — a domain that can be introduced or deferred without altering any other aggregate's internal design, since it only ever reads events other domains already produce.

Overall, the model is deliberately shaped around one architectural seam that recurs throughout: design-time artifacts (drafts, suggestions, unvalidated configuration) are always kept structurally distinct from run-time artifacts (published Flow Versions, active Deployments, CallSessions), with explicit, human-gated transitions between the two. This is the single most important invariant for the database design phase to preserve.

Step 9 — Review of the Domain Model

9.1 Missing Entities Identified During Review

- WebhookSubscription — tenants integrating their own CRM/EHR systems will need to subscribe to platform events (call ended, voicemail received); currently only the AI/reporting domains produce data, with no outbound event mechanism modeled.
- FlowTemplate — a reusable, industry-specific starting point (e.g., 'Hospital Appointment Flow') distinct from a Tenant's own IVRFlow; valuable for AI-assisted generation and for a future template marketplace.
- AgentPresenceLog — a time-series record of Employee status changes (Available/Busy/Offline), needed to compute accurate agent-utilization reports; currently only the current status is modeled on Employee, with no history.
- CallbackRequest — if callback-from-queue is implemented (see Future Enhancements in the SRS), this needs its own entity distinct from CallSession, since it represents a promise to call back rather than a call in progress.
- DataSubjectRequest — for GDPR/compliance-style erasure or access requests against caller data, particularly relevant to the bank/hospital verticals named in the Business Analysis.

9.2 Missing Services Identified During Review

- RateLimitingService — referenced implicitly by the SRS's toll-fraud/call-flooding NFRs but not yet modeled as a domain service; should sit alongside CallRoutingService and evaluate admission before a CallSession is created.
- WebhookDeliveryService — companion to the WebhookSubscription entity above, responsible for reliable outbound event delivery with retry/backoff.
- DataExportService — supports tenant offboarding (exporting all tenant data before deletion) and DataSubjectRequest fulfillment.

9.3 Missing Business Rules Identified During Review

- A CallSession must not be created for a Tenant whose Subscription usage has exceeded its concurrent-call or monthly-minutes entitlement (ties Billing & Metering into Call Routing, which the current model leaves implicit).
- A Role's Permission set must be validated against the Tenant's current plan entitlements not only at creation but re-validated on plan downgrade, to prevent privilege retention beyond what the new plan allows.
- An AlertRule that has fired must not re-notify for the same breach within a configurable cool-down window, to prevent notification storms during a sustained incident.

9.4 Architecture Improvements

The clean separation between IVR Design and IVR Execution (via the immutable Flow Version and the Deployment seam) is the strongest part of this model and should be preserved exactly as-is into the database schema — it is what prevents an in-progress edit from ever being able to affect a live call. The Aggregate boundaries drawn around FlowVersion (owning Node and Connection) and around CallQueue (owning QueueMembership) correctly reflect where atomic consistency is actually required; resist the temptation to further split these for perceived normalization purity, as doing so would reintroduce the risk of partially-consistent flow graphs or queue memberships becoming visible mid-transaction.

One area to reconsider before database design: CallSession currently owns AnalyticsEvent as a conceptual child, but AnalyticsEvent's actual write pattern (high-frequency, append-only, read mostly in aggregate) behaves more like a time-series/event-log store than a transactional child table. It is recommended to model AnalyticsEvent's persistence differently from the rest of the CallSession aggregate's fields even though the domain relationship (one CallSession, many events) remains accurate.

9.5 Scalability Concerns

- The CallSession and AnalyticsEvent write path is the platform's highest-volume, highest-concurrency data path and should be designed for horizontal write scaling independently of the lower-volume administrative aggregates (Tenant, IVRFlow, Department).
- The FlowVersion aggregate's immutability is scalability-friendly (safe to cache aggressively, safe to read-replicate) and should be leaned on: the Execution Engine should never need to hit a primary/write database for flow structure, only for CallSession state.
- Recording storage (audio files) must be modeled as separate, tenant-partitioned blob storage from day one, with only metadata in the relational store — recordings will be the fastest-growing storage category and the one most exposed to per-tenant volume variance (a hospital tenant vs. a small clinic).

9.6 Maintainability Concerns

- The Node/NodeConfiguration polymorphism (a single Node entity with a type discriminator and a variable-shape configuration value object) trades some database-level type safety for the ability to add new node types without schema migrations to a class-per-type table hierarchy — this is the right trade-off for an extensible IVR builder, but it does mean node-configuration validation must live firmly in the application layer (FlowValidationService), not be assumed enforceable by the database schema alone.
- Because the backend stack (Core Java Servlets, DAO pattern, no framework) will not enforce aggregate boundaries automatically, the DAO layer should be organized one-DAO-per-aggregate-root, with child entities (Node, Connection, QueueMembership, Subscription) accessed only through their owning aggregate's DAO — never given an independent DAO of their own — to keep the aggregate discipline intact in code, not just on paper.

9.7 Future Expansion Opportunities

- Multi-channel: the Node/Connection graph model generalizes naturally to SMS/WhatsApp/chat if a ChannelType is added to CallSession and Deployment, without touching the core aggregate structure.
- Outbound campaigns: an OutboundCampaign aggregate could reuse IVRFlow/FlowVersion as its logic definition, initiating CallSessions proactively rather than only in response to inbound DIDs.
- A/B testing: since Deployment already models 'which Flow Version is active where,' a weighted multi-version Deployment (e.g., 80/20 split) is a natural, low-disruption extension of the existing aggregate rather than a new concept.
- Skills-based and language-based routing can be deepened by promoting the Skill value object into a richer taxonomy entity if tenants need to manage large, tenant-specific skill catalogs.

This domain model, together with the entity attribute lists, aggregate boundaries, and relationship cardinalities documented above, is intended to be sufficient for a database architect to proceed directly to relational schema design without further clarification on the platform's core business semantics.